

Rapport TP3

Parallélisation par tâches OpenMP

5 mars 2026

Résumé

Ce rapport présente la parallélisation par **tâches OpenMP** de deux noyaux de calcul : (1) le calcul de π par la méthode Monte Carlo, et (2) le produit matrice creuse-vecteur (*SpMV*) en format CSR. Pour chaque noyau, nous décrivons l'implémentation, justifions les choix techniques, et analysons les performances mesurées.

Table des matières

1 Calcul de π par Monte Carlo

1.1 Q.1 – Programme parallèle à base de tâches OpenMP 3.0

1.1.1 Principe

On tire aléatoirement N points $(x, y) \in [0, 1]^2$. Un point est dans le quart de cercle si $x^2 + y^2 \leq 1$. La proportion de points dans le cercle tend vers $\frac{\pi}{4}$, d'où :

$$\pi \approx 4 \times \frac{\text{nb points dans le cercle}}{\text{nb total de points}}$$

1.1.2 Implémentation (Pi_MT/pi_sequentiel.c)

La parallélisation utilise des **tâches OpenMP** selon le schéma suivant :

```

1  #pragma omp parallel
2  {
3      #pragma omp single
4      {
5          for (int t = 0; t < nthreads * N_TASKS_PER_THREAD; t++)
6          {
7              #pragma omp task firstprivate(t)
8              {
9                  unsigned int seed = (unsigned int)(t * 31 + 2020);
10                 uint64_t local_count = 0;
11                 double x, y;
12
13                 for (int k = 0; k < N_TRIALS_PER_TASK; k++)
14                 {
15                     x = rand_r(&seed) / (double)RAND_MAX;
16                     y = rand_r(&seed) / (double)RAND_MAX;
17                     local_count += (((x * x) + (y * y)) <= 1.0);
18                 }
19
20                 #pragma omp atomic
21                 count += local_count;
22             }
23         }
24     }
25 }
```

1.1.3 Points clés de l'implémentation

Point clé

L'utilisation conjointe de `#pragma omp single`, `rand_r()` et `#pragma omp atomic` garantit à la fois la correction numérique (absence de *data race*) et de bonnes performances.

1. `#pragma omp single` : un seul thread crée l'ensemble des tâches; les autres threads les exécutent au fur et à mesure. Sans `single`, chaque thread créerait ses propres tâches, multipliant le travail.

2. **rand_r(&seed) au lieu de rand()** : la fonction **rand()** utilise un état global partagé et n'est pas thread-safe. **rand_r()** prend une graine locale en paramètre, éliminant toute *data race*.
3. **firstprivate(t)** : chaque tâche reçoit sa propre copie de **t** pour calculer une graine unique $\text{seed} = t * 31 + 2020$, garantissant des séquences aléatoires différentes.
4. **Variables x, y locales** à chaque tâche (déclarées dans le bloc **task**), évitant tout conflit entre threads.
5. **#pragma omp atomic** : accumulation thread-safe du compteur **count** sans utiliser de section critique (plus léger qu'un **critical**).
6. **Nombre total de tâches** = **nthreads** $\times$ **N_TASKS_PER_THREAD**, chacune effectuant **N_TRIALS_PER_TASK** tirages.

1.1.4 Résultats (4 threads, 1000 tâches/thread, 10 000 tirages/tâche)

Métrique	Valeur
Points totaux	40 000 000
Points dans le cercle	31 415 625
π approximé	3.141563
Valeur réelle (π)	3.141592...
Erreur relative	$\approx 9.4 \times 10^{-6}$
Temps d'exécution	0.34 s

TABLE 1 – Résultats du calcul de π par Monte Carlo (4 threads).

Remarque

L'approximation 3.141563 est très proche de la valeur réelle $\pi \approx 3.141593$ avec une erreur relative inférieure à 10^{-5} , ce qui confirme la convergence statistique de la méthode Monte Carlo.

2 Produit Matrice Creuse-Vecteur (SpMV)

2.1 Format CSR (*Compressed Sparse Row*)

Une matrice creuse de taille $N \times M$ avec NNZ éléments non nuls est représentée de manière compacte par trois tableaux :

- **values** (taille NNZ) : valeurs non nulles, stockées ligne par ligne.
- **JA** (taille NNZ) : indice de colonne de chaque valeur non nulle.

- IA (taille $N + 1$) : IA[i] est l'indice dans values/JA du premier élément de la ligne i ; IA[N] = NNZ.

Le nombre d'éléments non nuls de la ligne i est donc $\text{IA}[i+1] - \text{IA}[i]$, et les éléments sont aux indices $\text{IA}[i], \dots, \text{IA}[i+1]-1$ dans values et JA.

2.2 Q.2 — Produit matrice creuse-vecteur séquentiel

2.2.1 Implémentation (CSRMatrix.c)

```

1 void mult_CSR(CSRMatrix_t* A, double const* x, double* y)
2 {
3     int N = A->m_nrows;
4     uint64_t* ia = A->m_ia;
5     uint64_t* ja = A->m_ja;
6     double* val = A->m_values;
7
8     for (int i = 0; i < N; i++) {
9         double sum = 0.0;
10        for (uint64_t k = ia[i]; k < ia[i+1]; k++) {
11            sum += val[k] * x[ja[k]];
12        }
13        y[i] = sum;
14    }
15 }
```

2.2.2 Explication

Pour chaque ligne i :

1. On parcourt les éléments non nuls de cette ligne, indexés de $\text{ia}[i]$ à $\text{ia}[i+1]-1$.
2. Pour chaque élément k , on multiplie la valeur $\text{val}[k]$ par la composante correspondante du vecteur $\text{x}[\text{ja}[k]]$.
3. La somme est accumulée dans $\text{y}[i]$.

Le format CSR permet un accès séquentiel aux valeurs et aux indices de colonnes, ce qui est favorable au cache pour le parcours ligne par ligne.

2.2.3 Résultats séquentiels (grille 1000×1000 , 20 itérations)

2.3 Q.3 — Parallélisation par tâches OpenMP 4.0

2.3.1 Implémentation (CSRMatrix.c)

```

1 void mult_CSR_task(CSRMatrix_t* A, double const* x, double* y,
2                   int nb_tasks)
3 {
4     int N = A->m_nrows;
5     uint64_t* ia = A->m_ia;
6     uint64_t* ja = A->m_ja;
```

Métrique	Valeur
NROWS	1 000 000
NNZ	4 996 000
Densité	$\approx 0.5 \%$
NORME Y	3.70
Temps moyen / itération	6.9 ms
Performance	1 441 MFlops

TABLE 2 – Résultats du SpMV séquentiel (grille 1000×1000 , 20 itérations).

```

7  double*   val = A->m_values;
8  int block = (N + nb_tasks - 1) / nb_tasks;
9
10 #pragma omp parallel
11 {
12     #pragma omp single
13     {
14         for (int t = 0; t < nb_tasks; t++) {
15             #pragma omp task firstprivate(t)
16             {
17                 int istart = t * block;
18                 int iend   = istart + block;
19                 if (iend > N) iend = N;
20
21                 for (int i = istart; i < iend; i++) {
22                     double sum = 0.0;
23                     for (uint64_t k = ia[i]; k < ia[i+1]; k++) {
24                         sum += val[k] * x[ja[k]];
25                     }
26                     y[i] = sum;
27                 }
28             }
29         }
30     }
31 }
32 }
```

2.3.2 Explication

1. **Découpage en blocs** : les N lignes sont réparties en `nb_tasks` blocs de taille `block =  $\lceil N/\text{nb\_tasks} \rceil$` . Chaque tâche traite les lignes $[t \times \text{block}, (t+1) \times \text{block}[$.
2. `#pragma omp parallel + #pragma omp single` : un seul thread crée les tâches ; tous les threads (y compris le créateur) participent à leur exécution.
3. `#pragma omp task firstprivate(t)` : chaque tâche reçoit sa propre copie de `t` pour calculer ses bornes `istart/iend`.
4. **Pas de synchronisation sur y** : chaque tâche écrit dans un sous-ensemble disjoint de `y` (lignes différentes), il n'y a donc aucun conflit d'écriture.

5. **Surcharge de tâches** : on utilise `nb_tasks = 4 × nthreads` plutôt que `nthreads` pour améliorer l'équilibrage. Si certaines tâches terminent plus vite (lignes avec moins de non-zéros), les threads libérés prennent immédiatement les tâches restantes.

2.3.3 Résultats (grille 1000×1000 , 20 itérations, 4 threads)

Métrique	Séquentiel	Tâches OpenMP (4 threads)
NORME Y	3.70	3.70
Temps moyen	6.9 ms	6.1 ms
Performance	1 441 MFlops	1 634 MFlops
Speedup	1.0×	1.13×
Efficacité	100 %	28 %

TABLE 3 — Comparaison séquentiel vs. tâches OpenMP pour le SpMV.

Remarque

Les deux versions produisent la **même norme** pour le vecteur résultat ($\|y\| = 3.70$), ce qui valide la correction numérique de l'implémentation parallèle. Le speedup modeste ($1.13\times$ pour 4 threads) s'explique par la nature *memory-bound* du SpMV, analysée en section ??.

2.4 Q.4 — Avantages de la parallélisation par tâches

La parallélisation par tâches OpenMP du SpMV apporte plusieurs avantages par rapport à une parallélisation classique par `#pragma omp parallel for` :

2.4.1 1. Équilibrage dynamique de charge

Dans un Laplacien 2D, les lignes de bord ont 3–4 éléments non nuls tandis que les lignes intérieures en ont 5. Un découpage statique (par `schedule(static)`) peut créer un déséquilibre si les premiers ou derniers threads héritent de lignes plus légères. Avec les tâches, le runtime OpenMP distribue dynamiquement les tâches aux threads libres : dès qu'un thread termine une tâche, il en prend une nouvelle dans la file d'attente. En créant plus de tâches que de threads (surcharge), l'équilibrage est naturellement meilleur.

2.4.2 2. Flexibilité et composabilité

Les tâches permettent de composer facilement le parallélisme avec d'autres parties du programme. On peut imbriquer des tâches ou les combiner avec d'autres patterns (producteur-consommateur, pipelines) sans restructurer le code. C'est particulièrement utile dans un solveur itératif où le SpMV n'est qu'une étape parmi d'autres.

2.4.3 3. Granularité ajustable

Le paramètre `nb_tasks` permet de contrôler finement la granularité :

- **Peu de tâches** (= `nthreads`) : faible overhead mais équilibrage limité.
- **Beaucoup de tâches** ($\gg$ `nthreads`) : meilleur équilibrage mais overhead de création/ordonnancement plus élevé.

On peut ainsi trouver le compromis optimal pour chaque configuration matrice/architecture.

2.4.4 4. Compatibilité avec des matrices irrégulières

Pour des matrices issues de maillages non structurés (nombre de non-zéros très variable par ligne), l'approche par tâches est particulièrement adaptée car elle absorbe naturellement l'irrégularité de la charge.

2.4.5 Limites observées

Le speedup modeste ($1.13\times$ avec 4 threads, efficacité $\approx 28\%$) s'explique par le fait que le SpMV est un algorithme **memory-bound** : le facteur limitant n'est pas la puissance de calcul mais la **bande passante mémoire**.

Point clé

Chaque élément non nul requiert un accès indirect `x[ja[k]]` : les indices `ja[k]` sont irréguliers, ce qui provoque de nombreux *cache-miss*. Multiplier les threads n'augmente pas la bande passante disponible — c'est le *Roofline* mémoire qui plafonne les performances.

Le speedup réel dépend donc de la bande passante disponible par rapport au peak mémoire de l'architecture, plutôt que du nombre de cœurs.

3 Compilation et exécution

3.1 Calcul de π par Monte Carlo

```
cd Pi_MT
make clean && make
OMP_NUM_THREADS=4 ./pi.exe
```

Listing 1 – Compilation et exécution de Pi Monte Carlo

3.2 SpMV

```
cd SpMV
make clean && make
# Arguments : nx ny nb_iterations
```

```
OMP_NUM_THREADS=4 ./spmv.exe 1000 1000 20
```

Listing 2 – Compilation et exécution du SpMV

Les deux premiers arguments définissent les dimensions de la grille ($n_x \times n_y$) ; le troisième est le nombre d'itérations SpMV mesurées. La matrice générée est un Laplacien 2D de taille $N = n_x \times n_y$.

4 Conclusion

Ce TP illustre deux applications concrètes de la **parallélisation par tâches OpenMP** :

- **Monte Carlo** (π) : le modèle tâches est naturellement adapté — chaque tâche est indépendante, et l'accumulation atomique suffit pour garantir la correction. Le speedup est quasi-linéaire car l'algorithme est *compute-bound*.
- **SpMV** : la parallélisation est correcte (norme identique) mais le speedup reste limité ($1.13\times$) en raison du caractère *memory-bound* du noyau. Pour aller plus loin, il faudrait explorer des techniques de *blocking* ou de renumérotation des lignes pour améliorer la localité mémoire.

Les deux implémentations partagent le même patron : **parallel** $\rightarrow$ **single** $\rightarrow$ boucle de création de **task**, ce qui simplifie la portabilité et la maintenance du code.